



Reproducibility & Determinism at Scale

Seeds and sharding

- Set global and per-rank RNG seeds (Python, NumPy, framework, CUDA)
- Use a deterministic DistributedSampler so each rank gets a stable, unique shard

Deterministic ops

- Enable deterministic backends/flags
- Avoid or replace known nondeterministic kernels; document any exceptions

Floating-point non-associativity

- Different reduction orders/bucketization yield small numeric diffs
- Validate with tolerances (e.g., atol/rtol), not strict bitwise equality

Version and environment pinning

- Record container image, CUDA/cuDNN/NCCL, driver, framework versions
- Log code commit, configs/hyperparams, hardware topology and world size

Checkpoints

- Save model weights, optimizer state, RNG states (CPU/CUDA), and dataloader epoch/offset
- Include sharding metadata (DP/TP/PP groups, FSDP/ZeRO maps) for exact restarts
- Support elastic/changed world sizes with remapping info

Fault Tolerance, Checkpointing & Orchestration

Checkpointing:	- Save model, optimizer, LR scheduler, RNG states (CPU/CUDA), and sharding/group metadata - Write atomically to durable storage; verify and keep rolling N checkpoints
Failure handling:	- Synchronous jobs: relaunch and load latest checkpoint - Elastic jobs: surviving ranks continue; newcomers rendezvous, rebuild groups, and reshard from checkpoint
Launch/orchestration:	- Use SLURM/Kubernetes operators; set rank/world-size via env - Topology-aware placement (keep tightly coupled groups on same node/switch) - Add health checks, timeouts, exponential backoff, and automatic restart policies
LR & batch when elastic:	- Recompute effective global batch after world-size changes - Adjust learning rate accordingly; maintain gradient scaling and loss-scaling consistency - Log events and validate convergence after elastic resizes